TSwap Protocol Audit Report

Version 1.0

Ebukar

April 3, 2026

TSwap Protocol Audit Report

Ebukar

April 3, 2026

Prepared by: Ebukar

Lead Auditor: - Ebukar

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
  - Scope
  - Roles
- Executive Summary
  - Issues found
- Findings
  - High
    * [H-1] Incorrect fee calculation in `TSwapPool::getInputAmountBasedOnOutput` causes protocol to take too many tokens from users, resulting in lost fees
    * [H-2] Lack of slippage protection in TSwapPool::swapExactOutput causes users to potentially receive way fewer tokens
    * [H-3] `TSwapPool::sellPoolTokens` mismatches input and output tokens causing users to receive the incorrect amount of tokens

    * [H-4] In `TSwapPool::_swap` the extra tokens given to users after every `swapCount` breaks the protocol invariant of `x * y = k`
  - Medium
    * [M-1] `TSwapPool::deposit` is missing deadline check causing transactions to complete even after the deadline
  - Low
    * [L-1] `TSwapPool::LiquidityAdded` event has parameters out of order with the `TSwapPool::_addLiquidityAndMintTransfer` function that emits it
    * [L-2] Default value returned by `TSwapPool::swapExactInput` results in incorrect return value given
  - Informational
    * [I-1] `PoolFactory::PoolFactory__PoolDoesNotExist(address tokenAddress)` error is declared but not used and should be removed or used.
    * [I-2] `PoolFactory::constructor` Lacks zero address checks
    * [I-3] `PoolFactory::createPool` should use `.symbol()` instead of `.name()` to generate `liquidityTokenSymbol`
    * [I-4] Event is missing `indexed` fields

Protocol Summary

TSWapPool project is meant to be a permissionless way for users to swap assets between each other at a fair price. The protocol starts as simply a `PoolFactory` contract. This contract is used to create new "pools" of tokens. It helps make sure every pool token uses the correct logic. But all the magic is in each `TSwapPool` contract. You can think of each `TSwapPool` contract as it's own exchange between exactly 2 assets. Any ERC20 and the WETH token. These pools allow users to permissionlessly swap between an ERC20 that has a pool and WETH. Once enough pools are created, users can easily "hop" between supported ERC20s.

Disclaimer

The Ebukar team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

The findings described in this document corresponds with the following commit hash:

```
e643a8d4c2c802490976b538dd009b351b1c8dda
```

Scope

```
./src/
#-- PoolFactory.sol
#-- TSwapPool.sol
```

Roles

- Liquidity Providers: Users who have liquidity deposited into the pools. Their shares are represented by the LP ERC20 tokens. They gain a 0.3% fee every time a swap is made.
- Users: Users who want to swap tokens.

Executive Summary

Issues found

Severity	Number of issues found
High	4
Medium	1
Low	2
Info	4
Total	11

Findings

High

[H-1] Incorrect fee calculation in `TSwapPool::getInputAmountBasedOnOutput` causes protocol to take too many tokens from users, resulting in lost fees

Description: The `TSwapPoolgetInputAmountBasedOnOutput` function is intended to calculate the amount of tokens a user should deposit given an amount of tokens of output tokens. However, the function currently miscalculates the resulting amount. When calculating the fee, it scales the amount by 10_000 instead of 1_000.

Impact: Protocol takes more fees than expected from users. As a result, users swapping token via the `swapExactOutput` function will pay more than expected for their trades. This becomes particularly risky for users that provide infinite allowance to the `TSwapPool` contract. Moreover, note that the issue is worsened by the the fact that the `swapExactOutput` function does not allow users to specify a minimum of input tokens, as is describe in another issue in this report.

It's worth noting that the tokens paid by users are not lost, but rather can be swiftly taken by liquidity providers. Therefore, this contract could be used to trick users by having them swap their funds at unfavorable rates and finally rug pulling all liquidity from the pool.

Proof of Concept:

PoC

Include the following code in the `TSwapPool.t.sol` test suite

```
    function testSwapExactOutputIsFlawed() external {
        uint256 initialLiquidity = 100e18;

        // liquidityProvider deposits 100e18
```

```
        vm.startPrank(liquidityProvider);
        weth.approve(address(pool), initialLiquidity);
        poolToken.approve(address(pool), initialLiquidity);
        pool.deposit(initialLiquidity, 0, initialLiquidity, uint64(block.timestamp));
        vm.stopPrank();

        // User has 11 pool token balance
        address someUser = makeAddr("someUser");
        uint256 someUserStartingPoolTokenBalance = 11e18;
        poolToken.mint(someUser, someUserStartingPoolTokenBalance);

        // User buys 1 weth from the pool using pool token
        vm.startPrank(someUser);
        poolToken.approve(address(pool), type(uint256).max);
        pool.swapExactOutput(poolToken, weth, 1 ether, uint64(block.timestamp));
        vm.stopPrank();

        // initial liquidity was 1:1, so User should have paid ~1 pool token
        // however, it spent much more than that
        // User started with ~11 pool tokens but now have less than ~1 pool token after buying ~1 weth
        uint256 someUserEndingBalance = poolToken.balanceOf(someUser);
        assertLt(someUserEndingBalance, 1 ether);
        console2.log("User ending balance: ", someUserEndingBalance);

        // the liquidity provider can rug all funds in the pool now including those deposited by User.
        uint256 liquidityTokenBalance = pool.balanceOf(liquidityProvider);
        vm.prank(liquidityProvider);
        pool.withdraw(liquidityTokenBalance, 1, 1, uint64(block.timestamp));

        assertEq(poolToken.balanceOf(address(pool)), 0);
        assertEq(weth.balanceOf(address(pool)), 0);
        console2.log("Pool Token Balance: ", poolToken.balanceOf(address(pool)));
        console2.log("Weth Token Balance: ", weth.balanceOf(address(pool)));
    }
```

Recommended Mitigation:

```
    function getInputAmountBasedOnOutput(
        uint256 outputAmount,
        uint256 inputReserves,
        uint256 outputReserves
    )
```

```
        public
        pure
        revertIfZero(outputAmount)
        revertIfZero(outputReserves)
        returns (uint256 inputAmount)
    {
-       return((inputReserves * outputAmount) * 10000) / ((outputReserves - outputAmount) * 997);
+       return((inputReserves * outputAmount) * 1000) / ((outputReserves - outputAmount) * 997);
    }
```

[H-2] Lack of slippage protection in TSwapPool::swapExactOutput causes users to potentially receive way fewer tokens

Description: The `swapExactOutput` function does not include any sort of slippage protection. This function is similar to what is done in `TSwapPool::swapExactInput`, where the function specifies a `minOutputAmount`, the `swapExactOutput` function should specify a `maxInputAmount`.

Impact: If market conditions change before the transaction processes, the user could get a much worse swap.

Proof of Concept: 1. The price of 1 WETH right now is 1,000 USDC 2. User inputs a swapExactOutput looking for 1 WETH - inputToken = USDC - outputToken = WETH - outputAmount = 1 - deadline = whatever 3. The function does not offer a maxInput amount 4. As the transaction is pending in the mempool, the market changes! And the price moves HUGE -> 1 WETH is now 10,000 USDC. 10x more than the user expected 5. The transaction completes, but the user sent the protocol 10,000 USDC instead of the expected 1,000 USDC.

Recommended Mitigation: We should include a `maxInputAmount` so the user only has to spend up to a specific amount, and can predict how much they will spend on the protocol.

```
    function swapExactOutput(
        IERC20 inputToken,
+       uint256 maxInputAmount,
.
.
.
        inputAmount = getInputAmountBasedOnOutput(outputAmount, inputReserves, outputReserves);
+       if(inputAmount > maxInputAmount){
+           revert();
+       }
        _swap(inputToken, inputAmount, outputToken, outputAmount);
```

[H-3] `TSwapPool::sellPoolTokens` mismatches input and output tokens causing users to receive the incorrect amount of tokens

Description: The `sellPoolTokens` function is intended to allow users to easily sell pool tokens and receive WETH in exchange. Users indicate how many pool tokens they're willing to sell in the `poolTokenAmount` parameter. However, the function currently miscalculates the swapped amount. This is due to the fact that the `swapExactOutput` function is called, whereas the `swapExactInput` function is the one that should be called. Because users specify the exact amount of input tokens, not output.

Impact: Users will swap the wrong amount of tokens, which is a severe disruption of protocol functionality.

Proof of Concept:

PoC

Add the following to the `TSwapPool.t.sol` test suite

```
    function testSellPoolTokenMiscalculatesInputAmount() external {
        uint256 initialLiquidity = 100e18;

        // liquidityProvider deposits 100e18
        vm.startPrank(liquidityProvider);
        weth.approve(address(pool), initialLiquidity);
        poolToken.approve(address(pool), initialLiquidity);
        pool.deposit(initialLiquidity, 0, initialLiquidity, uint64(
            block.timestamp));
        vm.stopPrank();

        // user initially have 10 poolToken balance, extra ~40
            poolToken is given to user
        // user starts with ~50 poolToken balance
        uint256 extraBalance = 40e18;
        poolToken.mint(user, extraBalance);
        uint256 poolTokenAmount = 2e18;

        uint256 startingUserPoolTokenBalance = poolToken.balanceOf(user
            );
        uint256 expectedEndingUserPoolTokenBalance =
            startingUserPoolTokenBalance - poolTokenAmount;

        // user attempts to sell only ~2 poolToken for weth
        vm.startPrank(user);
        poolToken.approve(address(pool), type(uint256).max);
        weth.approve(address(pool), type(uint256).max);
        pool.sellPoolTokens(poolTokenAmount);
        vm.stopPrank();

```

```
        uint256 endingUserPoolTokenBalance = poolToken.balanceOf(user);

        // user ends up selling way more than the ~2 poolToken they wanted to and now has less than expected balance
        assertLt(endingUserPoolTokenBalance, expectedEndingUserPoolTokenBalance);
        console2.log("Expected User Pool Token Balance: ", expectedEndingUserPoolTokenBalance);
        console2.log("User Pool Token Balance: ", endingUserPoolTokenBalance)
    }
```

Recommended Mitigation: Consider changing the implementation to use `swapExactInput` instead of `swapExactOutput`. Note that this would also require changing the `sellPoolTokens` function to accept a new parameter (ie `minWethToReceive` to be passed to `swapExactInput`).

```
    function sellPoolTokens(
        uint256 poolTokenAmount,
+       uint256 minWethToReceive,
        ) external returns (uint256 wethAmount) {
-       return swapExactOutput(i_poolToken, i_wethToken, poolTokenAmount, uint64(block.timestamp));
+       return swapExactInput(i_poolToken, poolTokenAmount, i_wethToken, minWethToReceive, uint64(block.timestamp));
    }
```

Additionally, it might be wise to add a deadline to the function, as there is currently no deadline.

[H-4] In `TSwapPool::_swap` the extra tokens given to users after every `swapCount` breaks the protocol invariant of `x * y = k`

Description: The protocol follows a strict invariant `of x * y = k`. Where: - x: The balance of WETH - y: The balance of the pool token - k: The constant product of the two balances

This means, that whenever the balances change in the protocol, the ratio between the two amounts should remain constant, hence the `k`. However, this is broken due to the extra incentive in the `_swap` function. Meaning that over time the protocol funds will be drained.

The follow block of code is responsible for the issue.

```
    swap_count++;
    if (swap_count >= SWAP_COUNT_MAX) {
    swap_count = 0;
    outputToken.safeTransfer(msg.sender, 1_000_000_000_000_000_000);
    }
```

Impact: A user could maliciously drain the protocol of funds by doing a lot of swaps and collecting the extra incentive given out by the protocol. Most simply put, the protocol's core invariant is broken.

Proof of Concept:

1. A user swaps 10 times, and collects the extra incentive of `1_000_000_000_000_000_000` tokens
2. That user continues to swap until all the protocol funds are drained

   PoC

   Place the following into TSwapPool.t.sol.

```
    function testInvariantBroken() public {
        uint256 initialLiquidity = 100e18;

        vm.startPrank(liquidityProvider);
        weth.approve(address(pool), initialLiquidity);
        poolToken.approve(address(pool), initialLiquidity);
        pool.deposit(initialLiquidity, 0, initialLiquidity, uint64(block.timestamp));
        vm.stopPrank();

        uint256 outputWeth = 1e17;

        vm.startPrank(user);
        poolToken.approve(address(pool), type(uint256).max);
        poolToken.mint(user, 100e18);
        pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
        pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
        pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
        pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
        pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
        pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
        pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
        pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
        pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));

        int256 startingX = int256(weth.balanceOf(address(pool)));
        int256 expectedDeltaX = int256(outputWeth) * -1;

```

```
        pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.
            timestamp));
        vm.stopPrank();

        int256 endingX = int256(weth.balanceOf(address(pool)));
        int256 actualDeltaX = int256(endingX) - int256(startingX);

        assertEq(actualDeltaX, expectedDeltaX);
    }
```

Recommended Mitigation: Remove the extra incentive mechanism. If you want to keep this in, we should account for the change in the `x * y = k` protocol invariant. Or, we should set aside tokens in the same way we do with fees.

```
-   swap_count++;
-   // Fee-on-transfer
-   if (swap_count >= SWAP_COUNT_MAX) {
-       swap_count = 0;
-       outputToken.safeTransfer(msg.sender, 1_000_000_000_000_000_000)
    ;
-   }
```

Medium

[M-1] `TSwapPool::deposit` is missing deadline check causing transactions to complete even after the deadline

Description: The `deposit` function accepts a deadline parameter, which according to the documentation is "The deadline for the transaction to be completed by". However, this parameter is never used. As a consequence, operations that add liquidity to the pool might be executed at unexpected times, in market conditions where the deposit rate is unfavorable.

Impact: Transactions could be sent when market conditions are unfavorable, even when adding a deadline parameter.

Proof of Concept: The `deadline` parameter in unused in the `TSwapPool::deposit` function

Recommended Mitigation: Consider making the following changes to the function:

```
    function deposit(
        uint256 wethToDeposit,
        uint256 minimumLiquidityTokensToMint,
        uint256 maximumPoolTokensToDeposit,
        uint64 deadline
    )
        external
```

```
        revertIfZero(wethToDeposit)
+       revertIfDeadlinePassed(deadline)
        returns (uint256 liquidityTokensToMint)
    {...}
```

Low

[L-1] `TSwapPool::LiquidityAdded` event has parameters out of order with the `TSwapPool::_addLiquidityAndMintTransfer` function that emits it

Description: When the `LiquidityAdded` event is emitted in the `TSwapPool::_addLiquidityAndMintTran` function, it logs value in an incorrect order. The `poolTokenToDeposit` value should go in the third parameter position while the `wethToDeposit` value should go in the second parameter position

Impact: Event emmision is incorrect, which can lead to off-chain functions potentially malfunctioning

Proof of Concept:

The event emission parameter order:

```
    emit LiquidityAdded(msg.sender, poolTokensToDeposit, wethToDeposit);
```

differs from the event parameter order:

```
    event LiquidityAdded(
        address indexed liquidityProvider,
        uint256 wethDeposited,
        uint256 poolTokensDeposited
    );
```

Recommended Mitigation:

```
-   emit LiquidityAdded(msg.sender, poolTokensToDeposit, wethToDeposit);
+   emit LiquidityAdded(msg.sender, wethToDeposit, poolTokensToDeposit);
```

[L-2] Default value returned by `TSwapPool::swapExactInput` results in incorrect return value given

Description: The `swapExactInput` function is expected to return the actual amount of tokens bought by the caller. However, while it declares the named return value output it is never assigned a value, nor uses an explicit return statement.

Impact: The return value will always be 0, giving incorrect information to the caller.

Proof of Concept:

PoC

Add the following code to the `TSwapPool.t.sol` file

```
    function testSwapExactInputAlwaysReturnsZero(uint256 _inputAmount) external {
        uint256 initialLiquidity = 100e18;

        // liquidityProvider deposits 100e18
        vm.startPrank(liquidityProvider);
        weth.approve(address(pool), initialLiquidity);
        poolToken.approve(address(pool), initialLiquidity);
        pool.deposit(initialLiquidity, 0, initialLiquidity, uint64(block.timestamp));
        vm.stopPrank();

        // user sells 2 or more pool token
        uint256 inputAmount = bound(_inputAmount, 2, poolToken.balanceOf(user));
        uint256 minOutput = inputAmount / 2;

        // user calls the swapExactInput function to sell pool token and check return value
        vm.startPrank(user);
        poolToken.approve(address(pool), type(uint256).max);
        uint256 returnValue = pool.swapExactInput(poolToken, inputAmount, weth, minOutput, uint64(block.timestamp));
        vm.stopPrank();

        // return value is always 0
        assertEq(returnValue, 0);
    }
```

Recommended Mitigation: Make the following adjustment to the `TSwapPool::swapExactInput` function

```
    {
        uint256 inputReserves = inputToken.balanceOf(address(this));
        uint256 outputReserves = outputToken.balanceOf(address(this));

-       uint256 outputAmount = getOutputAmountBasedOnInput(inputAmount,inputReserves,outputReserves);
+       output = getOutputAmountBasedOnInput(inputAmount,inputReserves,outputReserves);

-       if (outputAmount < minOutputAmount) {
-           revert TSwapPool__OutputTooLow(outputAmount,
```

```
    minOutputAmount);}
+       if (output < minOutputAmount) {
+           revert TSwapPool__OutputTooLow(output, minOutPutAmount);
        }

-       _swap(inputToken, inputAmount, outputToken, outputAmount);
+       _swap(inputToken, inputAmount, outputToken, output);
    }
```

Informational

[I-1] `PoolFactory::PoolFactory__PoolDoesNotExist(address tokenAddress)` error is declared but not used and should be removed or used.

```
-    error PoolFactory__PoolDoesNotExist(address tokenAddress);
```

[I-2] `PoolFactory::constructor` Lacks zero address checks

```
    constructor(address wethToken) {
+       if (wethToken == address(0)), {
+           revert();
+       }
        i_wethToken = wethToken;
    }
```

[I-3] `PoolFactory::createPool` should use `.symbol()` instead of `.name()` to generate `liquidityTokenSymbol`

```
-    string memory liquidityTokenSymbol = string.concat("ts", IERC20(
    tokenAddress).name());
+    string memory liquidityTokenSymbol = string.concat("ts", IERC20(
    tokenAddress).symbol());
```

[I-4] Event is missing `indexed` fields

Index event fields make the field more quickly accessible to off-chain tools that parse events. However, note that each index field dsts extra gas during emission, so it's not necessarily best to index the maximum allowed per event (three fields). Each event should use three indexed fields if there are three or more fields, and gas usage is not particularly of concern for the events in question. If there are fewer than three fields, all of the fields should be indexed.

Found in src/PoolFactory.sol: Line: 35 Found in src/TSwapPool.sol: Line: 44 Found in src/TSwapPool.sol: Line: 46 Found in src/TSwapPool.sol: Line: 43